

Bank & Credit Card Statement Automation System

Internship Project Final Documentation & Technical Report

COMMIT & PUSH VERIFICATION: All codebase files, backend APIs, setup batch scripts, and frontend proxy configurations are 100% completed, committed, and pushed to remote repositories on branch `feature/initial-codebase`.

1. Project Metadata & Repository Information

- **Project Name:** Bank & Credit Card Statement Automation / Converter
- **Domain / Category:** Financial Data Extraction, OCR Parsing, ERP Integration, Full-Stack Web App
- **GitLab Repository (Primary Remote):** <https://gitlab.com/ankrok/bank-statement-automation.git>
- **GitHub Repository (Secondary Remote):** <https://github.com/praneetisingh/bank-statement-converter.git>
- **Active Working Branch:** `feature/initial-codebase`
- **Git Commit & Push Status:** Working tree clean. All commits pushed to 'origin/feature/initial-codebase' & 'github/feature/initial-codebase'.

2. Executive Summary & Task Overview

Manual data entry from bank and credit card statements into accounting software is error-prone and time-consuming. This project provides an end-to-end intelligent enterprise tool that extracts transaction rows from bank statements (PDFs, images) and transforms them into clean, structured accounting datasets.

Key Capabilities:

- **Multimodal Extraction:** High-accuracy extraction using direct PDF text parsing (PyMuPDF) and fallback OCR (Tesseract).
- **Configurable Layout:** Interactive UI allowing users to select required columns, rename headers, and reorder sequences.
- **Persistent Memory:** Automatically saves and loads mapping preferences in SQLite based on Client & Account Suffix.
- **Multi-Format Export & ERP Sync:** Exports styled Excel files (.xlsx), QuickBooks IIF files (.iif), and direct API sync to QBO, NetSuite, Xero, Zoho.

3. Architecture & Technology Stack

- **Frontend:** React 18, Vite dev server, Vanilla CSS with Glassmorphism theme, Lucide icons.
- **Backend:** Python 3.10+, FastAPI framework, Uvicorn server, Pydantic data schemas.
- **Parsing & Storage:** PyMuPDF (fitz), pytesseract OCR, SQLite3 (statements.db), openpyxl.

4. Directory Structure

```
bank-statement-automation/
├── backend/
│   ├── main.py           # FastAPI endpoints & CORS configuration
│   ├── parser.py         # Multimodal extraction engine & OCR logic
│   ├── db.py             # SQLite database interface for mapping configs
│   ├── excel_generator.py # Openpyxl logic for styled Excel downloads
│   ├── erp_connectors.py # QuickBooks IIF & Cloud ERP API connectors
│   ├── requirements.txt  # Backend Python dependencies
│   └── statements.db     # Persistent SQLite database file
├── frontend/
│   ├── src/ (App.jsx, App.css, index.css)
│   ├── package.json, vite.config.js, Dockerfile
├── run_setup_and_start.bat # One-click Windows launch script
└── PROJECT_DOCUMENTATION.md # Project documentation file
```

5. How to Run the Application

Option A: One-Click Windows Launch

1. Install Python 3.10+ (check 'Add Python to PATH') and Node.js 18+.
2. Double-click **run_setup_and_start.bat** in the project root folder.
3. The app will auto-configure and open at **http://localhost:5173/**.

Option B: Manual Setup

- Backend: `cd backend && python main.py` (Port 8000)
- Frontend: `cd frontend && npm run dev` (Port 5173)

6. REST API Endpoints Reference

Method	Endpoint	Description
POST	/api/upload	Parses statement PDF/image, extracts transactions & mapping config.
POST	/api/save-config	Saves column mapping configuration in SQLite database.
POST	/api/export-excel	Generates and streams styled Excel workbook (.xlsx).
POST	/api/export-iif	Generates QuickBooks Desktop IIF import file.
POST	/api/sync-erp	Syncs transaction payload directly to NetSuite, QBO, Xero, or Zoho.